CpSc 8400: Design and Analysis of Algorithms

Instructor: Dr. Brian Dean — Fall 2024
Handout 3: Programming Assignment #4 — Online / Coursera

This assignment is described in a text document rather than a video, since we intend to rotate through different problems here across different semesters.

This assignment is intended to give you practice applying various optimization heuristics to find reasonably good solutions to a prominent problem known to be "hard" to solve optimally. It is designed to be somewhat open-ended, since a wide range of approaches might be viable (you are encouraged to work together and to attend office hours if you want to share ideas on what methods tend to work the best).

Linear Arrangement

The goal of this problem is to order n elements of data so that similar elements end up near each-other. Equivalently, let's represent our data by nodes in a graph / network, with edges connecting between pairs of nodes that represent similar data elements. In this case, we want to find a linear arrangement of the nodes in our graph, where edges tend to span only short distances. A common objective here is to minimize the sum of squared lengths of all the edges — the so-called "squared wire length" objective. More precisely, we want to assign each node i to a position x_i on the number line so as to minimize

$$\sum (x_i - x_j)^2,$$

where the sum is taken over all edges (i, j). The positions $x_0 \dots x_{n-1}$ of the n nodes must be some permutation of $\{0, 1, \dots, n-1\}$. That is, the nodes are to be mapped to positions $0, 1, \dots, n-1$ on a number line in some optimal order.

Figure 1 shows an example of a small problem instance on just 4 nodes, along with two possible orderings; the first has objective value $1^2 + 1^2 + 1^2 + 3^2 = 12$, which is better than the second, with objective value $1^2 + 2^2 + 2^2 + 3^2 = 18$.

This problem has numerous applications. For example, it often arises in data visualization or clustering, and it can also help with storage performance (e.g., by improving cache performance by storing similar elements near each-other).

The problem above is unfortunatle NP-hard, so as with many other prominent problems, it is not known how to compute an *optimal* solution in an efficient manner. It will be interesting to see the quality of solutions you, as a class, mange to compute, using combinations of heuristics outlined in the current module (e.g., greedy algorithms, relaxation and rounding, iterative refinement, etc.).

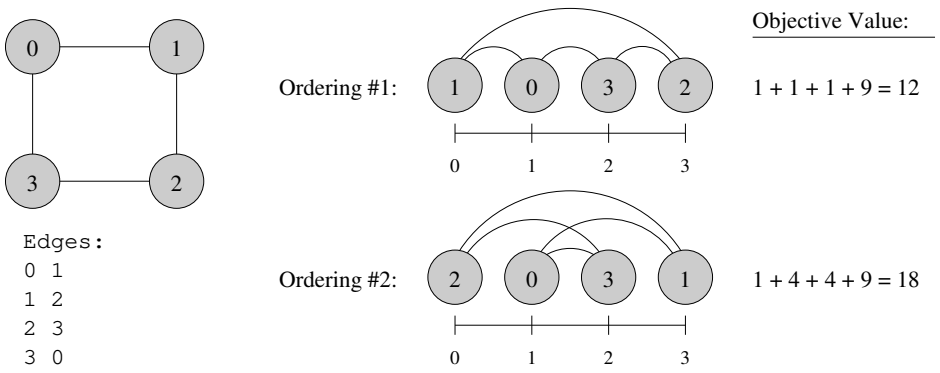

Figure 1: Example instance with 4 nodes and 4 edges, with two possible orderings, one better than the other.

Evaluation Details

Your program will be run on 12 test cases, and your score will be based on solution quality — if your solution yields and objective twice as large as the target solution we have in mind (usually the best solution we have found so far), your score for that test case will be 50%. A solution at least as good as our target will earn a score of 100% for that test case.

Your program will be allowed the same amount of time to run for each test case: 5 seconds for C/C++, 10 seconds for Java, and 28 seconds for Python. You may therefore want to incorporate a timer in your code to have it print the best solution found so far right before the time limit runs out. However, note that most functions that report timing are expensive to call, so you want to be careful to check timing only sporadically, otherwise you will waste precious time with these calls.

The template provided should be relatively easy to follow. There is a `get_neighbors` function that returns a vector of the neighbors of a particular node (i.e., a list of the indices of the nodes that are similar to it). There is also a version of this function provided for local testing. Note that it is possible for some nodes to have zero neighbors. Hopefully, it should be obvious how these nodes can be dealt with so they don't compromise the quality or efficiency of your solution.

Of the 12 test cases provided, the first is the same as the local test case, and as such receives zero weight. The first ten test cases have increasing sizes of $n \leq 1000$, with $n = 5000$ for case 11 and $n = 10{,}000$ for case 12.